




# Lecture 8 : Edge Impulse, une plateforme MLOps pour le TinyML

## Des données au firmware, reproductible et multi-cibles, projet EcoTri 4.0

*Un nouveau client, de nouveaux déchets : peut-on reconstruire et redéployer le modèle en une journée ?*

**Pr. Aissam BERRAHOU**

Département Ingénierie des systèmes embarqués et mobiles, Smart Systems Lab
IA en Edge Computing, projet fédérateur EcoTri 4.0, version 2027
ENSIAS, Université Mohammed V de Rabat

aissam.berrahou@ensias.um5.ac.ma

# Feuille de route

Pourquoi une plateforme ? | Données et impulse | Apprendre : classer, détecter | Optimiser : EON | Déployer et maintenir

- **Pourquoi une plateforme ?** : *Qu'est-ce qui coûte cher dans un projet TinyML, au-delà du modèle ?*
- **Données et impulse** : *Comment ingérer, étiqueter et décrire une chaîne de traitement reproductible ?*
- **Apprendre : classer, détecter** : *Transfer learning, FOMO et détection d'anomalies : quand utiliser quoi ?*
- **Optimiser : EON** : *Comment l'EON Tuner et l'EON Compiler respectent-ils un budget RAM / Flash / latence ?*
- **Déployer et maintenir** : *Une impulse, plusieurs cibles, des versions et une API (jalon J8).*

# Objectifs

Ce que vous saurez faire en sortant de la salle

**À la fin de cette lecture, vous serez capables de :**

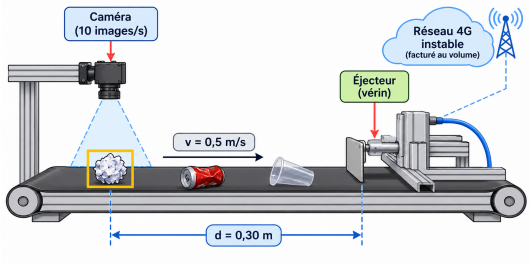
- **Situer** Edge Impulse dans la boucle MLOps du TinyML et en **critiquer** les limites (dépendance cloud, boîte noire partielle).
- **Concevoir** une *impulse* : entrée, bloc de traitement, bloc d'apprentissage, et justifier chaque paramètre.
- **Expliquer** l'architecture et la fonction de perte de **FOMO** ; **choisir** entre classification, FOMO et détection d'anomalie.
- **Lire** un résultat de l'EON Tuner comme un **front de Pareto** et **chiffrer** le gain de l'EON Compiler.
- **Déployer** la même impulse sur la Portenta (OpenMV) et la XIAO (Arduino), et **versionner** le projet.

## Le livrable : jalon J8

Projet Edge Impulse versionné « EcoTri-Vision », détecteur FOMO déployé sur deux cartes, rapport EON Tuner argumenté (précision, RAM, Flash, latence mesurée contre estimée).

# Le problème déclencheur

## Un deuxième centre de tri, d'autres emballages, une semaine pour livrer




- Le modèle de la lecture 5 a demandé **trois semaines** de scripts Python, de conversions TFLite et de réglages à la main.
- Le client B veut aussi **compter** et **localiser** les objets (plusieurs objets par image).

### À vous (5 min, par équipe)

Listez les étapes à refaire pour ce nouveau site. Lesquelles sont **automatisables** ? Lesquelles exigent votre **jugement d'ingénieur** ?

! le modèle n'est qu'une petite partie du système (Sculley et al., 2015). Une plateforme MLOps outille **tout le reste**.

# Pourquoi une plateforme ?

*Qu'est-ce qui coûte cher dans un projet TinyML, au-delà du modèle ?*

# Pourquoi une plateforme ?

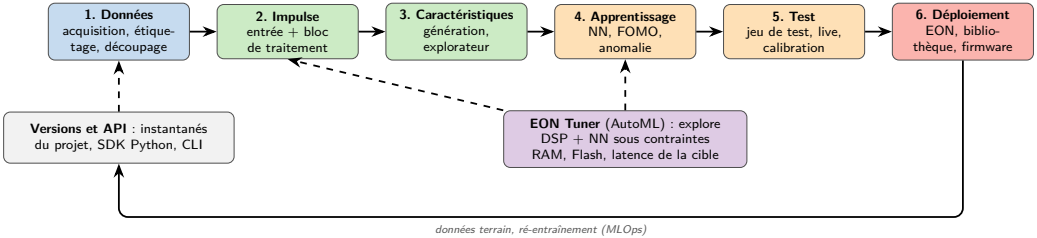
Les cinq défis du TinyML (Hymel, Banbury et al., MLSys 2023, Harvard / Edge Impulse)

| Défi | Constat | Réponse de la plateforme |
|---|---|---|
| Données | collecte hétérogène, étiquetage coûteux | ingestion multi-sources, étiquetage assisté |
| Pile logicielle | Python, TF, C/C++, SDK constructeur | chaîne unique navigateur → cible |
| Matériel | dizaines d'architectures, mémoires différentes | estimation RAM / Flash / latence par cible |
| Modèle | hyperparamètres contraints par la cible | AutoML : EON Tuner |
| Déploiement, suivi | pas de MLOps unifié | versions, API, SDK Python, CLI |

! Edge Impulse ne remplace pas la théorie des lectures 4 et 5 : il l'**industrialise**. Vous devez savoir **ce qu'il fait** pour savoir **quand le contredire**.

# Pourquoi une plateforme ?

## Le flux de travail Edge Impulse




**Lecture :** chaque étape correspond à un onglet du Studio ; le pointillé montre où l'EON Tuner intervient ; la boucle du bas est la boucle MLOps de la lecture 7.

### Ce que l'on gagne

- Reproductibilité : l'impulse est une **spécification** complète (DSP + NN + seuils).
- Estimations de coût **avant** déploiement, par carte.
- Code C++ portable, sans dépendance, généré automatiquement.
- Partage d'équipe, projets publics clonables (travaux de Rovai, UNIFEI).

### Ce qu'il faut surveiller

- Les données vont au **cloud** : vérifier la confidentialité (lecture 1, ENF-3).
- Estimations $\neq$ mesures : toujours **mesurer sur la cible**.
- Offre gratuite limitée (temps de calcul, taille du projet).
- Dépendance fournisseur : exporter le `.tflite` et les données.

### Règle du cours

Toute valeur annoncée par le Studio doit être confrontée à une mesure sur la carte (temps, `gc.mem_free()`, `micros()`).

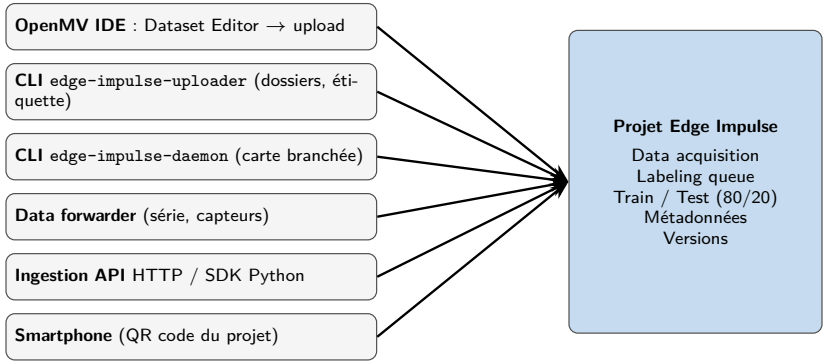
# Données et impulse

*Comment ingérer, étiqueter et décrire une chaîne de traitement reproductible ?*

# Données et impulse

## Six portes d'entrée pour les données

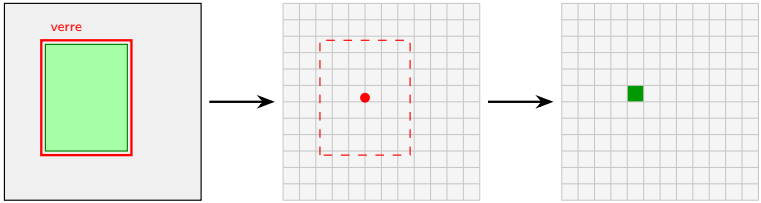





! capturer avec **le capteur de déploiement** (Vision Shield monochrome, OV2640 de la XIAO) : sinon décalage de domaine dès le premier jour.

# Données et impulse






(a) boîte englobante annotée (b) centre de la boîte (c) cible : 1 cellule « verre »

*les 143 autres cellules sont « fond » : déséquilibre fort ⇒ perte pondérée*

- Mode « Bounding boxes » du projet ; file d'étiquetage avec **suivi d'objets** entre images et pré-étiquetage par un modèle (YOLO).
- FOMO ne garde que le **centre** : la taille est perdue.

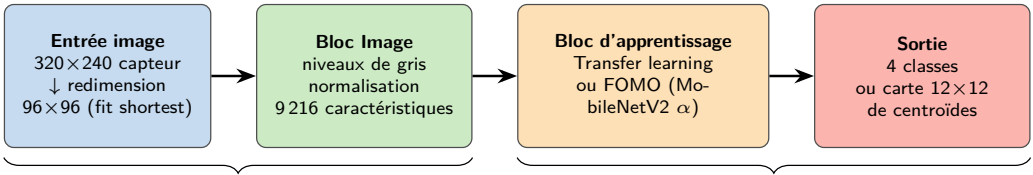
## Ordre de grandeur

Pour 4 classes : 100 à 200 images par classe, plusieurs objets par image, positions et éclairages variés, 20 % en test.

# Données et impulse

## L'impulse : une chaîne déterministe

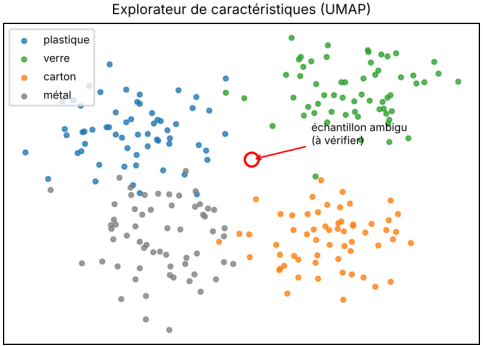
**Impulse = chaîne déterministe entrée → traitement → modèle**




- **Resize mode** : *squash* (déforme), *fit shortest* (rogne), *fit longest* (bandes) ; à reproduire à l'identique sur la cible.
- Pour une image $W \times H \times C$, le vecteur d'entrée a $W\,H\,C$ composantes : $96{\times}96{\times}1 = \mathbf{9\,216}$, $96{\times}96{\times}3 = 27\,648$.

# Données et impulse

## L'explorateur de caractéristiques : diagnostiquer avant d'entraîner




- Projection 2D (UMAP, McInnes et al., 2018) des caractéristiques.
- Pour les images, les caractéristiques sont celles d'un réseau pré-entraîné : les grappes annoncent la **séparabilité**.
- Un point isolé dans une mauvaise grappe : souvent une **erreur d'étiquette**.

> **!** nettoyer les données avant de toucher aux hyperparamètres : c'est le levier le moins cher (Ng, « data-centric AI »).

**Question flash**

?

Vous entraînez en RGB 96×96, mais la Portenta capture en niveaux de gris 320×240 et le script redimensionne en *squash*. Citez deux incohérences et leur effet.

**Question flash**

?

Vous entraînez en RGB 96×96, mais la Portenta capture en niveaux de gris 320×240 et le script redimensionne en *squash*. Citez deux incohérences et leur effet.

**Réponse**

(1) Canaux : le modèle attend 27 648 entrées, la cible en fournit 9 216 : erreur ou entrée dupliquée sans sens. (2) Redimensionnement différent de l'entraînement : objets déformés, chute de précision silencieuse. La règle : **même prétraitement** à l'entraînement et sur la cible.

# Apprendre : classer, détecter

*Transfer learning, FOMO et détection d'anomalies : quand utiliser quoi ?*

# Apprendre : classer, détecter

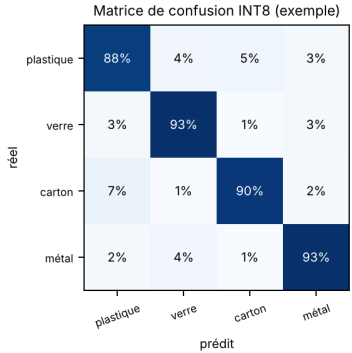
## Trois tâches, trois blocs d'apprentissage



| Bloc | Question | Sortie | Dans EcoTri |
|---|---|---|---|
| Transfer learning (images) | *quel* matériau ? (un objet) | $K$ probabilités | Portenta, classifieur matériau |
| Object detection : FOMO | *combien*, *où* ? | centroïdes + classes | XIAO, comptage sur le tapis |
| Object detection : MobileNet SSD, YOLO | où, *quelle taille* ? | boîtes | trop lourd pour nos MCU |
| Anomaly (K-means, GMM, Visual AD) | est-ce *inhabituel* ? | score | objet inconnu, moteur (lecture 4) |

! partir de la **question métier** et de la **sortie** exigée par l'éjecteur, puis vérifier le budget mémoire.

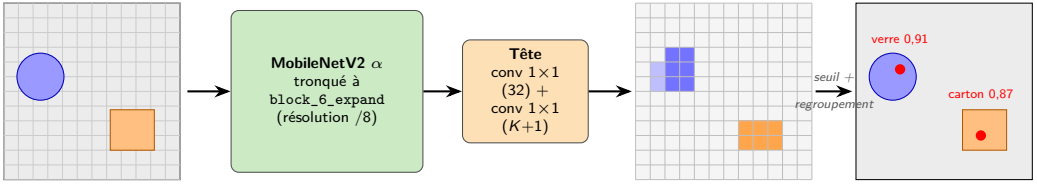
- Base MobileNetV1/V2 pré-entraînée (ImageNet), largeur $\alpha \in \{0{,}05\,;\,0{,}1\,;\,0{,}35\,;\,1{,}0\}$.
- Coût en MAC d'une convolution séparable $\propto \alpha^2$ (Howard et al., 2017) : $\alpha = 0{,}35$ coûte $\approx 8\times$ moins que $\alpha = 1$.
- Réglages : nombre d'époques, taux d'apprentissage, neurones de la tête, *dropout*, augmentation (retournement, luminosité, rognage).




Matrice de confusion INT8 : le carton est confondu avec le plastique (barquettes).

# Apprendre : classer, détecter

FOMO : *Faster Objects, More Objects* (Moreau et Kelcey, Edge Impulse, 2022)



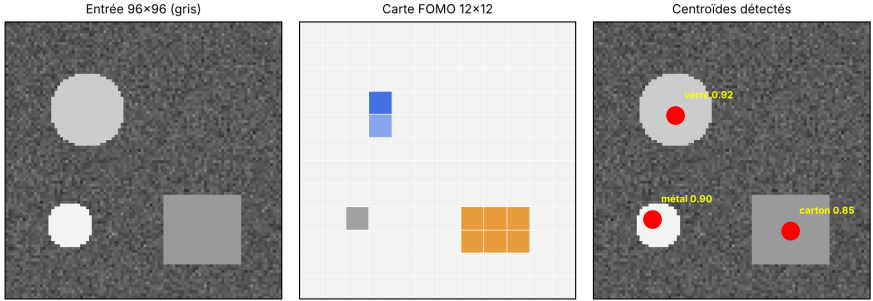

- Détection reformulée en **classification par cellule** d'une grille, entièrement convolutive.
- Facteur 8 : 96×96 → 12×12 ; 160×160 → 20×20.

## Ordres de grandeur (documentation EI)

Moins de 200 Ko de RAM ; jusqu'à 30× plus rapide que MobileNet SSD ; 30 img/s sur Nicla Vision (Cortex-M7).

# Apprendre : classer, détecter

## FOMO en action : de l'image à la décision




- Chaque cellule produit $K+1$ scores (le fond est la classe 0) ; on garde les cellules au-dessus du seuil, puis on **fusionne** les cellules voisines de même classe.
- Le centroïde en pixels : $x = (j + 0{,}5) \cdot 8$, $y = (i + 0{,}5) \cdot 8$, puis mise à l'échelle vers l'image du capteur.

Entropie croisée par cellule, pondérée pour compenser la domination du fond :

$$\mathcal{L} = -\frac{1}{N} \sum_{i,j} \sum_{k=0}^{K} w_k \, y_{ijk} \, \log \hat{p}_{ijk}, \qquad w_0 \ll w_{k \geq 1}$$

- Métrique : **F1** sur les centroïdes (une détection est juste si elle tombe dans la bonne cellule).
- Le fond représente $\approx 99$ % des cellules : la précision brute n'a aucun sens.

## Hypothèses de FOMO

- objets de **taille similaire** ;
- objets **séparés** d'au moins une cellule (8 px), sinon fusion ;
- pas de taille ni de boîte précise en sortie.

> **!** objets trop proches : augmenter la résolution ou réduire le facteur de la carte.

**Question flash**

?

Entrée FOMO 160×160 en niveaux de gris, 4 classes + fond. Taille de la carte de sortie ? Combien de valeurs ? Deux canettes à 10 px l'une de l'autre sont-elles séparées ?

**Question flash**

?

Entrée FOMO 160×160 en niveaux de gris, 4 classes + fond. Taille de la carte de sortie ? Combien de valeurs ? Deux canettes à 10 px l'une de l'autre sont-elles séparées ?

**Réponse**

Carte 20×20, soit $20 \times 20 \times 5 = \mathbf{2\,000}$ scores. 10 px correspondent à un peu plus d'une cellule (8 px) : elles seront le plus souvent dans deux cellules adjacentes de même classe, donc **fusionnées** par le regroupement. Il faut un seuil de fusion plus strict ou une carte plus fine.

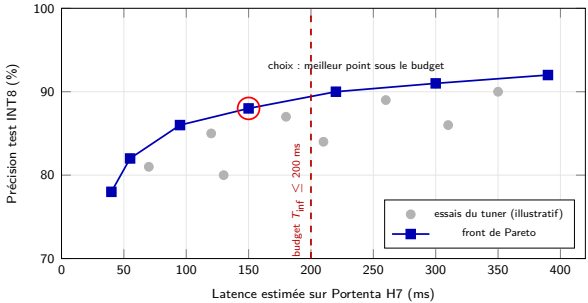
# Optimiser : EON

*Comment l'EON Tuner et l'EON Compiler respectent-ils un budget RAM / Flash / latence ?*

# Optimiser : EON

## L'EON Tuner : AutoML sous contraintes matérielles

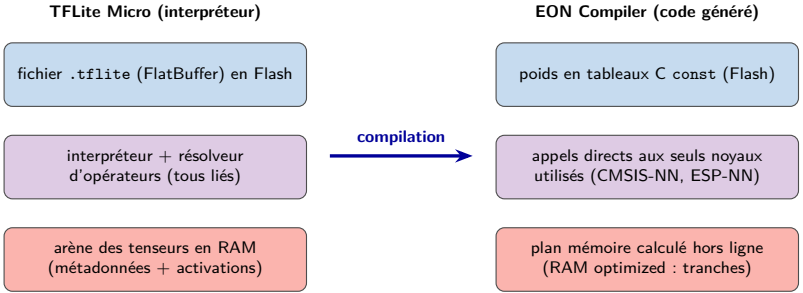
- Espace de recherche : bloc DSP (taille, couleur) $\times$ architecture NN ($\alpha$, couches, époques).
- Recherche **aléatoire** (Bergstra et Bengio, 2012) + estimation rapide du coût sur la cible choisie.
- Contraintes : latence max, RAM, Flash ; espace personnalisable (*search space*).




c'est une **NAS** bon marché : on lit le résultat comme un front de Pareto et l'on choisit le point dominant qui respecte le budget de la station.

# Optimiser : EON

## L'EON Compiler : supprimer l'interpréteur




*doc. Edge Impulse : jusqu'à 25–65 % de RAM et 10–35 % de Flash en moins, même précision*

TFLite Micro (David et al., MLSys 2021) lit le modèle à l'exécution ; EON génère du C++ spécialisé : moins de code lié, plan mémoire calculé hors ligne. L'option **RAM optimized** recalcule des valeurs plutôt que de les stocker (léger surcoût de latence).

# Optimiser : EON

Faire le budget complet d'une impulse

$$\mathrm{RAM}_{\mathrm{tot}} = \mathrm{RAM}_{\mathrm{img}} + \mathrm{RAM}_{\mathrm{DSP}} + \mathrm{RAM}_{\mathrm{NN}} + \mathrm{RAM}_{\mathrm{app}}$$

$$T = T_{\mathrm{capt}} + T_{\mathrm{DSP}} + T_{\mathrm{NN}} + T_{\mathrm{post}}$$

- Le Studio estime $\mathrm{RAM}_{\mathrm{DSP}}$, $\mathrm{RAM}_{\mathrm{NN}}$, $T_{\mathrm{DSP}}$, $T_{\mathrm{NN}}$ pour la carte choisie.
- Vous ajoutez le tampon image, la pile radio et le post-traitement.

Exemple : XIAO, FOMO 96×96 gris, $\alpha = 0{,}35$

Image QVGA RGB565 : 150 Ko (PSRAM) ; caractéristiques : 9 Ko ; NN (EON) : $\approx$ 120 Ko ; Wi-Fi/BLE : $\approx$ 80 Ko. Total SRAM interne $\approx$ **210 Ko** sur 512 Ko : marge confortable.

Piège classique

Le FOMO $\alpha = 0{,}35$ peut saturer la mémoire d'un ESP32 sans PSRAM : revenir à $\alpha = 0{,}1$ (recommandation du forum Edge Impulse).

**Question flash**

?

Le Studio annonce pour la Portenta : DSP 4 ms, NN 92 ms (EON), 118 Ko de RAM. Sur la carte vous mesurez 155 ms. Proposez trois causes et la mesure qui permet de trancher.

**Question flash**

**?** Le Studio annonce pour la Portenta : DSP 4 ms, NN 92 ms (EON), 118 Ko de RAM. Sur la carte vous mesurez 155 ms. Proposez trois causes et la mesure qui permet de trancher.

**Réponse**

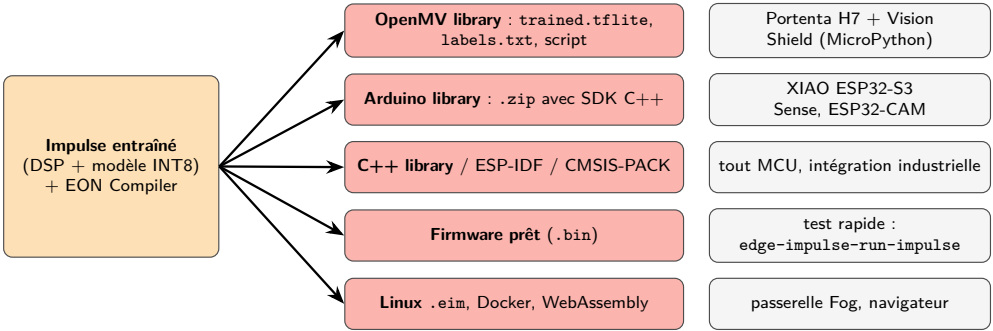
(1) Capture et redimensionnement non comptés : chronométrer `snapshot()` seul. (2) Moteur différent : OpenMV utilise TFLite Micro, pas le code EON : comparer avec l'Arduino library. (3) Fréquence, cache ou modèle chargé en SDRAM externe : mesurer avec `load_to_fb`. On isole chaque terme du budget $T$.

# Déployer et maintenir

*Une impulse, plusieurs cibles, des versions et une API (jalon J8).*

# Déployer et maintenir

## Une impulse, plusieurs cibles




Pour OpenMV, seuls les modèles **INT8** sont acceptés ; la Portenta n'a que des images en niveaux de gris.

# Déployer et maintenir

## Sur la Portenta : bibliothèque OpenMV (firmware 4.5 et suivants, module `ml`)

```
import sensor, ml, math, image
sensor.reset(); sensor.set_pixformat(sensor.GRAYSCALE)
sensor.set_framesize(sensor.B320X320)
sensor.set_windowing((240, 240))
net = ml.Model("trained.tflite", load_to_fb=True)
labels = [l.rstrip() for l in open("labels.txt")]
SEUIL = [(math.ceil(0.5 * 255), 255)]

def fomo_post(model, inputs, outputs):
    _, oh, ow, oc = model.output_shape[0]
    sx, sy = inputs[0].roi[2]/ow, inputs[0].roi[3]/oh
    res = [[] for _ in range(oc)]
    for k in range(oc):   # une image de scores par classe
        c = image.Image(outputs[0][0, :, :, k] * 255)
        for b in c.find_blobs(SEUIL, area_threshold=1,
      pixels_threshold=1):
            res[k].append((int(b.cx()*sx), int(b.cy()*sy)))
    return res

while True:
    img = sensor.snapshot()
    for k, d in enumerate(net.predict([img], callback=fomo_post)):
        if k: print(labels[k], d)          # k = 0 : fond
```

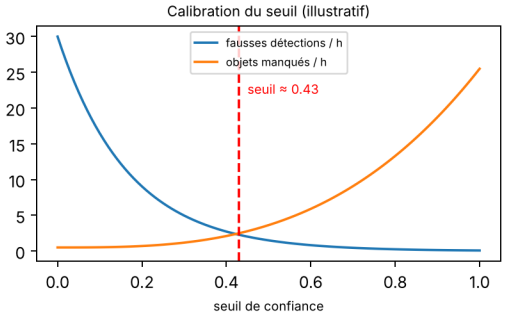
- Copier `trained.tflite` et `labels.txt` sur le disque de la carte.
- `load_to_fb` place le modèle dans la mémoire du frame buffer (à tester si RAM insuffisante).
- Le script `ei_object_detection.py` fourni utilise la même logique : `find_blobs` fait seuillage et regroupement.

# Déployer et maintenir

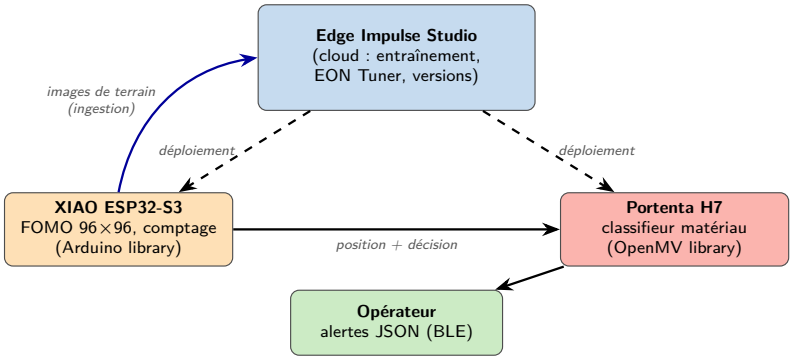
## Sur la XIAO : bibliothèque Arduino (SDK C++)

```
#include <EcoTri-Vision_inferencing.h>
signal_t sig;
sig.total_length = EI_CLASSIFIER_INPUT_WIDTH
                 * EI_CLASSIFIER_INPUT_HEIGHT;
sig.get_data = &lire_pixels;   // depuis le tampon caméra
ei_impulse_result_t r = {0};
EI_IMPULSE_ERROR e = run_classifier(&sig, &r, false);
for (size_t i = 0; i < r.bounding_boxes_count; i++) {
  auto b = r.bounding_boxes[i];
  if (b.value == 0) continue;
  Serial.printf("%s (%.2f) x=%u y=%u\n",
                b.label, b.value, b.x, b.y);
}
Serial.printf("DSP %d ms, NN %d ms\n",
  r.timing.dsp, r.timing.classification);
```

- `run_classifier()` exécute **toute l'impulse** (DSP + NN + post-traitement FOMO).
- `r.timing` donne les temps **mesurés** : à comparer aux estimations du Studio.
- Activer la PSRAM (*OPI PSRAM*) et ESP-NN pour la XIAO.




- **Model testing** sur le jeu de test jamais vu ; **live classification** depuis la carte.
- **Performance calibration** : choisir le seuil selon le coût relatif des fausses détections et des oublis.
- **Versions** : instantané daté des données, de l'impulse et des résultats ; restaurable (lecture 7 : reproductibilité).
- **API et SDK Python** : automatiser ingestion, entraînement, profilage (`ei.model.profile`) dans une CI.




La XIAO localise et compte (FOMO), la Portenta décide du matériau ; les images de terrain reviennent alimenter le projet.

**Question flash**

?

Le client B impose : 4 matériaux, comptage, $T \leq 250$ ms, aucune image ne quitte l'usine après la mise en service. Proposez une impulse (taille, couleur, bloc), une cible, et dites comment respecter la contrainte de confidentialité avec une plateforme cloud.

**Question flash**

?

Le client B impose : 4 matériaux, comptage, $T \leq 250$ ms, aucune image ne quitte l'usine après la mise en service. Proposez une impulse (taille, couleur, bloc), une cible, et dites comment respecter la contrainte de confidentialité avec une plateforme cloud.

**Réponse**

FOMO 96×96 gris, MobileNetV2 $\alpha = 0{,}35$, INT8, EON, sur la XIAO (Arduino library) ; validation du budget par mesure de `r.timing`. Le cloud ne sert qu'à l'**entraînement** sur un jeu collecté avec accord et anonymisé (pas d'opérateurs dans le champ) ; en production l'inférence est locale et seules des étiquettes sortent. Pour le ré-entraînement : collecte ponctuelle validée ou offre sur site (on-premise).

# À retenir

- **Edge Impulse** est une plateforme MLOps : données, impulse, apprentissage, optimisation, déploiement, versions.
- Une **impulse** = prétraitement + modèle, identique à l'entraînement et sur la cible.
- **FOMO** : détection par classification de cellules (facteur 8), centroïdes sans taille, objets séparés et de taille proche.
- **EON Tuner** : recherche aléatoire contrainte, à lire comme un front de Pareto. **EON Compiler** : code C++ spécialisé, moins de RAM et de Flash.
- **Mesurer** sur la carte : les estimations ne remplacent jamais la mesure.
- **Versionner** et **automatiser** (API, SDK) : c'est ce qui rend le modèle maintenable.

# Glossaire et références

| Terme | Définition courte |
|---|---|
| Impulse | chaîne entrée + DSP + apprentissage |
| FOMO | détection par grille de centroïdes |
| EON Tuner | AutoML contraint par la cible |
| EON Compiler | compilation du modèle en C++ |
| F1 | moyenne harmonique précision/rappel |
| UMAP | projection 2D non linéaire |


- S. Hymel, C. Banbury et al. (Harvard / Edge Impulse), « Edge Impulse : An MLOps Platform for Tiny Machine Learning », MLSys 2023, arXiv :2212.03332.
- R. David et al., « TensorFlow Lite Micro », MLSys 2021 (Google / Harvard).
- V. Janapa Reddi et al., *Machine Learning Systems*, Harvard, mlsysbook.ai ; labs Edge Impulse de M. Rovai (UNIFEI, TinyML4D).
- A. Howard et al., « MobileNets », 2017 ; M. Sandler et al., « MobileNetV2 », CVPR 2018.
- L. Moreau, M. Kelcey, « FOMO : Faster Objects, More Objects », Edge Impulse, 2022.
- J. Bergstra, Y. Bengio, « Random Search for Hyper-Parameter Optimization », JMLR 2012.
- D. Sculley et al., « Hidden Technical Debt in ML Systems », NeurIPS 2015.
- D. Situnayake, J. Plunkett, *AI at the Edge*, O'Reilly, 2023.
- Documentation Edge Impulse (FOMO, EON, OpenMV) ; Arduino, « Training a Custom ML Model for Portenta H7 ».


## Prochaine étape

**Tutorial 8** : premier projet Edge Impulse de bout en bout avec la Portenta. **TP 8** : détecteur FOMO multi-cibles et jalon J8.